

Too hot to handle

Just-in-time compilation for effects and handlers.

MARCIAL GAISSERT, University of Tübingen, Germany
CARL FRIEDRICH BOLZ-TEREICK, University of Düsseldorf, Germany
JONATHAN IMMANUEL BRACHTHÄUSER, University of Tübingen, Germany

Effect systems are a programming language feature that has gained in popularity recently. However, most implementations still incur significant performance overhead for stack operations. Tracing JITs are an approach to just-in-time compilation that uses dynamic execution traces. On microbenchmarks, we show that, by leveraging the existing RPython toolkit for generating tracing jits to implement a bytecode interpreter, many stack operations are optimized away. This causes significant speedups, also when comparing to a backend compiling directly to machine code ahead-of-time via LLVM.

CCS Concepts: • **Software and its engineering** → **Control structures; Abstraction, modeling and modularity**; • **Theory of computation** → **Type theory**.

Additional Key Words and Phrases: effect systems, effect polymorphism, second-class values, capabilities

ACM Reference Format:

Marcial Gaißert, Carl Friedrich Bolz-Tereick, and Jonathan Immanuel Brachthäuser. 2022. Too hot to handle: Just-in-time compilation for effects and handlers.. *Proc. ACM Program. Lang.* 6, OOPSLA1, Article 76 (April 2022), 8 pages. <https://doi.org/10.1145/3527320>

1 INTRODUCTION

Introduction with references

- Effekt (lexical effect handlers)
- Pypy/RPython (tracing JITs)

1.1 Compilation of Effekt

How to get from Effekt to jit bytecode

- evidence (reference lift-inference)

2 FORMAL PRESENTATION

Explain the syntax and semantics

- Stack of stacks
- Constructs left out of the formal presentation?
- Unenforced restriction: Evidence corresponds to metastack

Since — thanks to lift inference — we always know how many markers on the stack we would have to skip in a canonical semantic representation, it is natural to represent the stack in a segmented fashion. This way, instead of searching for the k th marker, we can “just” access the k th stack segment. In effect, we have a stack of stacks. To distinguish the two levels, we call the outer level stack the “metastack”, in analogy to metacontinuations.  useful reference for metacontinuations, maybe some additional relation to them. Where canonical representations would push some marker on the stack, we start a new stack. In notation, this looks quite similar if we annotate the stack bottom symbol with the marker.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

© 2022 Copyright held by the owner/author(s).
2475-1421/2022/4-ART76
<https://doi.org/10.1145/3527320>

```

// TODO Running example
effect Emit(i: Int): Unit
effect Abort(): Bottom

def reversed[R]() { body:  $\Rightarrow$  R/Emit }
= try {
  body()
} with Emit { (i: Int)  $\Rightarrow$ 
  val r = resume(());
  do Emit(i); r
}

def delayEmit[R]() { body:  $\Rightarrow$  R/Emit }
= reversed[R]{ reversed[R]{ body } }

def main() = try { delayEmit {
  do Emit(2);
  do Emit(3);
  if (2 < 3) {
    do Abort()
  }
}} with Emit { (i: Int)  $\Rightarrow$ 
  println(i); resume()
} with Abort { ()  $\Rightarrow$  () }

(a) in Effekt

```

```

l14: "main_419"
  num1  $\leftarrow$  1
  ptr0  $\leftarrow$  new stack l1()
  push stack ptr0
  ptr0  $\leftarrow$  new {m0  $\mapsto$  l13}(num0)
  ptr1  $\leftarrow$  new {m0  $\mapsto$  l12}()
  push l5()
  num0  $\leftarrow$  num0  $\oplus$  num1
  num1  $\leftarrow$  0
  num2  $\leftarrow$  0
  ptr1  $\leftarrow$  new {m0  $\mapsto$  l11}(ptr1)
  ... register movements
  jump l16
l15: "?generated"
  num2  $\leftarrow$  num2  $\oplus$  num0
  num0  $\leftarrow$  num3  $\oplus$  num0
  ... register movements
  jump l21
l16: "delayEmit_418"
  num3  $\leftarrow$  0
  ptr0  $\leftarrow$  new {m0  $\mapsto$  l15}(num0, num1, ptr0)
  ... register movements
  jump l21
l21: "reversed_415"
  num0  $\leftarrow$  1
  ptr2  $\leftarrow$  new stack l17()
  push stack ptr2
  ptr1  $\leftarrow$  new {m0  $\mapsto$  l20}(num2, ptr1)
  num0  $\leftarrow$  num1  $\oplus$  num0
  num1  $\leftarrow$  0
  ... register movements
  ptr1.m0(num0, num1, ptr0)

(b) in JIT bytecode

```

Fig. 1. Running example program

■ Overview of the Effekt compiler (?)

Fig. 2

In the presented calculus, the computations of evidence and the management of the metastack are mostly independent, and this corresponds to the implementation. However, they will always be closely related syntactically. That is, there are two kinds of uses for push stack: One corresponds to a reset operation, and occurs together with a new stack and the instantiation of matching evidence. The other corresponds to a resume, and is directly followed by a return. This restriction is not enforced by the implementation, which could allow for additional static optimizations. ■ Really?

3 IMPLEMENTATION

- General remarks, RPython
 - Expand
 - Update code snippets, rules

Program Syntax:

- Remove last column (just for own documentation)
- Should push stack return evidence?

Program	$P ::= \vec{B}_i$
Blocks	$B ::= \{ l(\vec{x}_i) : s \}$ l
Statements	$s ::= x \leftarrow v; s$ $\vec{x}_i \leftarrow \text{primitive } f(\vec{x}_j); s$ $x \leftarrow 0; s \mid x \leftarrow 1; s$ $x_1 \leftarrow x_2 \oplus x_3; s$ jump B if x_1 then $B(\vec{x}_i)$ else s push $B(\vec{x}_i); s$ return $(\vec{x}_i)$ $x_1 \leftarrow \text{shift } x_2; s$ $x \leftarrow \text{shift } v; s$ $x \leftarrow \text{new stack } B(\vec{x}_i); s$ push stack $x; s$ $x \leftarrow \text{new } V(\vec{x}_j); s$ $x.m(\vec{x}_i)$ $x \leftarrow t(\vec{x}_i); s$ $x \xrightarrow{\text{match } \{t(\vec{x}_i) \Rightarrow B\}}$ $x_1 \leftarrow x_2; s$ $x_1 \leftrightarrow x_2; s$ drop $x; s$
VTables	$V ::= \{ \vec{m}_i \mapsto \vec{B}_i \}$
Labels and tags	l, t, m
Variables	x

Stack syntax:

Stacks	$k ::= \#_\ell$ $B(\vec{v}_i) :: k$
Metastacks	$M ::= \circ$ $k :: M$

Abstract machine syntax:

Abstract machine	$m ::= \langle P \mid X \mid s \mid M \rangle$ $\langle P \mid X \mid s \mid M \leftarrow P M \rangle \leftarrow P$ $\langle P \mid X \mid s \mid M \rightarrow P M \rangle \rightarrow P$
Register file	$X ::= \{ \vec{x}_i \mapsto \vec{v}_i \}$

programs	Program
basic blocks	Block
block labels	
literals / constants	CONST_*
primitive calls	PRIM_OP
evidence zero / one	CONST_INT
evidence composition	ADD
jumps:	
jump	JUMP
conditional jump	IFZERO
stack management:	
push a frame	PUSH
return	RETURN
metastack management:	
capturing stacks	SHIFT_DYN
capturing stacks	SHIFT
creating new stacks	NEW_STACK
pushing a stack	PUSH_STACK
codata:	
object creation	NEW
method invocation	INVOKE
data:	
data construction	CONSTRUCT
matching	MATCH
register management:	
copying	COPY
swapping	SWAP
dropping	DROP
virtual table	VTable

empty stack
stack frame
empty metastack
stack

abstract machine tuple
(unwinding)
(rewinding)

Fig. 3. Syntax of the jit bytecode.

Many of the abstract machine rules correspond directly to the RPython implementation. For example, the rule

Reductions without program or stack context:

$$\begin{aligned}
(\text{const}) \quad & \langle X \mid x \leftarrow v; s \rangle \longrightarrow \langle X\{x \mapsto v\} \mid s \rangle \\
(\text{prim}) \quad & \langle X \mid \vec{x}_i \leftarrow \text{primitive } f(\vec{x}_j); s \rangle \longrightarrow \langle X\{\vec{x}_i \mapsto \vec{v}_i\} \mid s \rangle \quad \text{if } f(\overrightarrow{X(\vec{x}_j)}) = \vec{v}_i \\
(\text{evi here}) \quad & \langle X \mid x \leftarrow 0; s \rangle \longrightarrow \langle X\{x \mapsto 0\} \mid s \rangle \\
(\text{evi there}) \quad & \langle X \mid x \leftarrow 1; s \rangle \longrightarrow \langle X\{x \mapsto 1\} \mid s \rangle \\
(\text{evi add}) \quad & \langle X \mid x_1 \leftarrow x_2 \oplus x_3; s \rangle \longrightarrow \langle X\{x_1 \mapsto v\} \mid s \rangle \quad \text{if } v = X(x_2) \oplus X(x_3)
\end{aligned}$$

Register Management

$$\begin{aligned}
(\text{copy}) \quad & \langle X \mid x_1 \leftarrow x_2; s \rangle \longrightarrow \langle X\{x_1 \mapsto v\} \mid s \rangle \quad \text{if } v = X(x_2) \\
(\text{swap}) \quad & \langle X \mid x_1 \leftrightarrow x_2; s \rangle \longrightarrow \langle X\{x_1 \mapsto v_1, x_2 \mapsto v_2\} \mid s \rangle \quad \text{if } v_1 = X(x_1), v_2 = X(x_2) \\
(\text{drop}) \quad & \langle X \mid \text{drop } x; s \rangle \longrightarrow \langle X|_{\text{dom}(X) - x} \mid s \rangle \quad \text{if } v = X(x_2)
\end{aligned}$$

Data and codata

$$\begin{aligned}
(\text{new}) \quad & \langle X \mid x \leftarrow \text{new } V(\vec{x}_j); s \rangle \longrightarrow \langle X\{x \mapsto \langle V, \vec{v}_j \rangle\} \mid s \rangle \quad \text{if } v_j = X(x_j) \\
(\text{invoke}) \quad & \langle X \mid x.m(\vec{x}_i) \rangle \longrightarrow \langle \{\vec{x}_k \mapsto \vec{v}_k\} \mid s' \rangle \quad \text{if } \vec{v}_k = \overrightarrow{X(x_i)}, \vec{v}_j \\
& \quad \text{and } \{l(\vec{x}_k) : s'\} = V(m) \\
& \quad \text{and } X(x) = \langle V, \vec{v}_j \rangle \\
(\text{cons}) \quad & \langle X \mid x \leftarrow t(\vec{x}_i); s \rangle \longrightarrow \langle X\{x \mapsto \langle t, \vec{v}_i \rangle\} \mid s \rangle \quad \text{if } v_i = X(x_i) \\
(\text{match}) \quad & \langle X \mid x \text{ match } \{t_c(\vec{x}_{ic}) \Rightarrow B_c\} \rangle \longrightarrow \langle X\{\vec{x}_{ic} \mapsto \vec{v}_i\} \mid \text{jump } B_c \rangle \quad \text{if } t_c = t \text{ and } X(x) = \langle t, \vec{v}_i \rangle
\end{aligned}$$

Jumps

$$\begin{aligned}
(\text{jump}) \quad & \langle X \mid \text{jump } \{l(\vec{x}_j) : s\} \rangle \longrightarrow \langle X|_{\vec{x}_j} \mid s \rangle \\
(\text{if } t) \quad & \langle X \mid \text{if } x_1 \text{ then } \{l(\vec{x}'_i) : s_1\}(\vec{x}_i) \text{ else } s_2 \rangle \longrightarrow \langle \{\vec{x}'_i \mapsto \vec{v}_i\} \mid s_1 \rangle \quad \text{if } v_i = X(x_i) \\
& \quad \text{and } X(x_1) = \top \\
(\text{if } f) \quad & \langle X \mid \text{if } x_1 \text{ then } \{l(\vec{x}'_i) : s_1\}(\vec{x}_i) \text{ else } s_2 \rangle \longrightarrow \langle X \mid s_2 \rangle \quad \text{if } X(x_1) = \perp
\end{aligned}$$

Metastack management

$$\begin{aligned}
(\text{new stack}) \quad & \langle X \mid x \leftarrow \text{new stack } B(\vec{x}_j); s \rangle \longrightarrow \langle X\{x \mapsto B(\vec{v}_j) :: \#_\ell :: \circ\} \mid s \rangle \quad \text{if } v_j = X(x_j) \\
(\text{shift dyn}) \quad & \langle X \mid x_1 \leftarrow \text{shift } x_2; s \rangle \longrightarrow \langle X \mid x_1 \leftarrow \text{shift } v; s \rangle \quad \text{if } v = X(x_2)
\end{aligned}$$

Reductions with stack, but without program context :

$$(\text{kcong}) \quad \langle X \mid s \mid k :: M \rangle \longrightarrow \langle X' \mid s' \mid k :: M \rangle \quad \text{if } \langle X \mid s \rangle \longrightarrow \langle X' \mid s' \rangle$$

Classical stack management

$$\begin{aligned}
(\text{push}) \quad & \langle X \mid \text{push } B(\vec{x}_j); s \mid k :: M \rangle \longrightarrow \langle X \mid s \mid B(\vec{v}_j) :: k :: M \rangle \quad \text{if } v_j = X(x_j) \\
(\text{Mret}) \quad & \langle X \mid \text{return } (\vec{x}_i) \mid \#_\ell :: M \rangle \longrightarrow \langle X \mid \text{return } (\vec{x}_i) \mid M \rangle \\
(\text{ret}) \quad & \langle X \mid \text{return } (\vec{x}_i) \mid \{l(\vec{x}_j) : s\}(\vec{v}_k) :: k :: M \rangle \longrightarrow \langle \{\vec{x}_j \mapsto \vec{v}_j\} \mid s \mid k :: M \rangle \quad \text{for } \vec{v}_j = \overrightarrow{X(x_i)}, \vec{v}_k
\end{aligned}$$

Metastack management

$$\begin{aligned}
(\text{shift}) \quad & \langle X \mid x \leftarrow \text{shift } v; s \mid M \rangle \longrightarrow \langle X \mid x \leftarrow \text{shift } v; s \mid \circ \Leftarrow M \rangle \Leftarrow \\
(\text{unwind } 1) \quad & \langle X \mid x \leftarrow \text{shift } (v \oplus 1); s \mid M \Leftarrow k :: M' \rangle \Leftarrow \longrightarrow \langle X \mid x \leftarrow \text{shift } v; s \mid k :: M \Leftarrow M' \rangle \Leftarrow \\
(\text{unwind } 0) \quad & \langle X \mid x \leftarrow \text{shift } 0; s \mid M' \Leftarrow k' :: M \rangle \Leftarrow \longrightarrow \langle X\{x \mapsto k' :: M'\} \mid s \mid M \rangle \\
(\text{push stack}) \quad & \langle X \mid \text{push stack } x; s \mid M \rangle \longrightarrow \langle X \mid s \mid M' \Leftarrow M \rangle \Leftarrow \quad \text{if } M' = X(x) \\
(\text{rewind } 1) \quad & \langle X \mid s \mid k' :: M' \Leftarrow M \rangle \Leftarrow \longrightarrow \langle X \mid s \mid M' \Leftarrow k' :: M \rangle \Leftarrow \\
(\text{rewind } 0) \quad & \langle X \mid s \mid \circ \Leftarrow M \rangle \Leftarrow \longrightarrow \langle X \mid s \mid M \rangle
\end{aligned}$$

Handling of program context:

$$\begin{aligned}
(\text{Pcong}) \quad & \langle P \mid X \mid s \mid M \rangle \longrightarrow \langle P \mid X' \mid s' \mid M' \rangle \quad \text{if } \langle X \mid s \mid M \rangle \longrightarrow \langle X' \mid s' \mid M' \rangle \\
(\text{lookup}) \quad & \langle P \mid X \mid s \mid M \rangle \longrightarrow \langle P \mid X \mid s[l \mapsto B] \mid M \rangle \quad \text{if } B = \{l(\vec{x}_i) : s\} \in P
\end{aligned}$$

Fig. 4. Abstract machine semantics for the jit bytecode.

(new) $\langle X \mid x \leftarrow \text{new } V(\vec{x}_j); s \rangle \longrightarrow \langle X\{x \mapsto \langle V, \vec{v}_j \rangle\} \mid s \rangle$ if $v_j = X(x_j)$
 directly corresponds to the following code:

```

elif isinstance(op, NEW):
    env.set_codata(op.out, CoData.make(op.args, env, op.vtable))

```

Looking at the rules

$$\begin{aligned}
 (\text{shift}) \quad & \langle X \mid x \leftarrow \text{shift } v; s \mid M \rangle \longrightarrow \langle X \mid x \leftarrow \text{shift } v; s \mid \circ \leftarrow M \rangle \leftarrow p \\
 (\text{unwind } 1) \quad & \langle X \mid x \leftarrow \text{shift } (v \oplus 1); s \mid M \leftarrow p k \vdash M' \rangle \leftarrow p \longrightarrow \langle X \mid x \leftarrow \text{shift } v; s \mid k \vdash M \leftarrow p M' \rangle \leftarrow p \\
 (\text{unwind } 0) \quad & \langle X \mid x \leftarrow \text{shift } 0; s \mid M' \leftarrow p k' \vdash M \rangle \leftarrow p \longrightarrow \langle X \{ x \mapsto k' \vdash M' \} \mid s \mid M \rangle
 \end{aligned}$$

we have the following implementation:

```

elif isinstance(op, SHIFT_DYN):
    n = promote(env.get_int(op.n))
    (cont, metastack) = split_metastack_n(MetaStack(stack, None, metastack), n+1)
    env.set_cont(op.out, cont)
    record_exact_value(metastack is not None, True)
    if metastack is not None:
        stack = metastack.stack
        metastack = metastack.tail
    else:
        stack = None
    context = get_context(pc_block, pc_instruction, stack, metastack)

```

This just merges *(unwind 0)* and *(unwind 1)* into the implementation as a function call to make the code more readable. Additionally, it provides the jit with the (statically ensured) invariant that we will not try to capture more stack than we have.

■ can_enter_jit heuristics

3.1 Optimizations

For all of the below:

- Maybe we should measure the performance effect of those (i.e. compare with vs without)

3.1.1 *Specializing by environment shape.* ■ Box and pointer diagrams?

Stack frames, data values, and codata values (i.e., objects) all internally have to store an environment. Since we cannot have RPython data types with inlined arrays, and the sizes of those environments are not known at the compile time of the implementation, this results in allocations for 2 arrays — one for numbers and one for references. This is undesirable for common cases like, e.g., the cons cells of a list. Thus, for all of those, we have specialized variants for common cases, namely for:

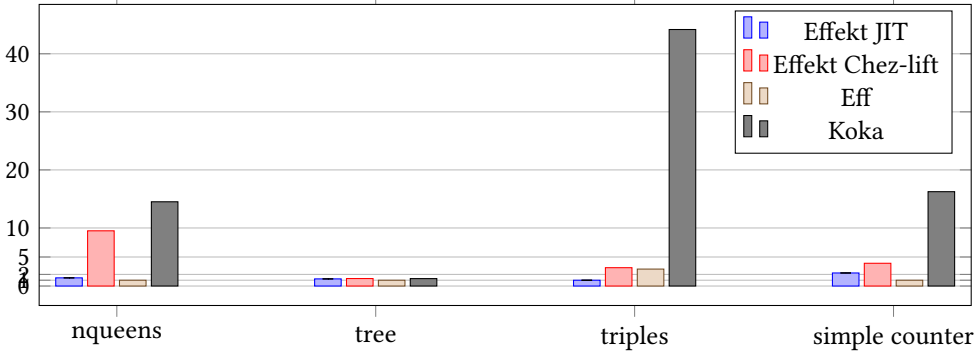
■ List of how much we specialize

To prevent dynamic dispatch for those specialized versions, we declare the correct specialization to the jit using program information. We can easily get at those in all cases by caching the frame type by return program counter (for stacks), caching codata sizes by virtual table (for codata), or using the form of the match clause (for data).

3.1.2 Additional JIT entry points. RPython requires us to call into the JIT at some program points in a way that ensures that we do this at least once in every loop. An easy heuristic is to do this on every (textually) backwards transfer of control.

For breaking all loops (and thus, all hot ones), it would be enough to just allow entering the JIT on those control transfer operations. However, for RPython's allocation removal to work well, it is desirable to cut the loop *before* values are allocated and *after* they are used. Thus, we additionally enter the jit after each push (which allocates a stack frame).

3.1.3 Fake loops. RPython would usually detect a loop whenever it reaches the same (smaller) program location again, in this case whenever we return to the same location. If we now, e.g., call



(a) Plot of execution times as factors of fastest measured time, lower is better.

benchmark	Effekt		Effekt		Effekt		Effekt	
	JIT	±	Chez-lift	Chez-lift (interpreted)	RPython interpreter	Eff	Koka	
nqueens	1132	2	7622	7579	723000	801.5	11629	
tree	235.2	0.6	245.7	250.8	16645	190.9	243.6	
triples	47.9	1.3	151.4	154.6	3840	140.1	2116	
simple counter	304.2	4.5	529.8	522.1	17042	135.1	2195	

(b) Table of execution times in milliseconds.

Fig. 5. Execution times of the benchmarks.

a library function twice in a row, we will potentially trace a false loop that will never execute more than once. This problem is well known for tracing jits [Reference]. To alleviate it, it is useful to consider some of the calling context. Thus, we also consider the target of the topmost stack frame when determining the program location. Note that, since we do not push for tail calls, this corresponds to a distinction about the immediate

4 EVALUATION

Threats to validity

4.1 Quantitative Evaluation of Performance

We evaluated the combined Effekt implementation with the JIT backend against different implementations of languages with effect handlers. For this, we used a set of community-developed microbenchmarks found at [Reference effect-handlers-bench].

Hardware and system information

Figure 5 shows the results of benchmarking the implementation on those benchmarks.

Re-Check the following with the final measurement results

In all benchmarks, we are within a factor of at most 2.25 of the best implementation available for that benchmark. For triples, we are best by a factor of 2.92. Thus, the resulting backend is competitive with the state-of-the-art.

Describe measurements Test nqueens for JIT: 1132 ms

Quantitative Evaluation - Figure and table with times - Compare vs others regarding time

4.2 Qualitative Evaluation of Performance

Qualitative Evaluation / Result - Sketch why certain patterns are ("always") removed by the JIT

```

p4 = (p1: Environment).values_num
i6 = p4[0]
i8 = p4[1]
i10 = p4[2]
i12 = p4[3]
i14 = p4[4]
i16 = p4[5]
i18 = p4[6]
i20 = p4[7]
i22 = p4[8]
p23 = (p1: Environment).values_ptr
p25 = p23[0]
p27 = p23[1]
p29 = p23[2]
p31 = p23[3]
p33 = p23[4]
p35 = p23[5]
Label 5641224384:
l_u bar_415 +1 push l_0()
guard not invalidated
l_u bar_415 +2 num_0 ← 0
l_u bar_415 +3 num_0 ← num_0 ⊕ num_0
l_u bar_415 +4 ptr_1 ← l_0()
...
l_u bar_415 +8 ptr_1.m_0(num_0, ptr_0)
p36 = (p25: CoData_arb_arb).vtable
guard p36 == ConstPtr(ptr37)
i38 = (p25: CoData_2_2).values_num_0
i39 = (p25: CoData_2_2).values_num_1
p40 = (p25: CoData_2_2).values_ptr_0
p41 = (p25: CoData_2_2).values_ptr_1
l_u ?generated +0 num_1 ← 0
l_u ?generated +1 num_0 ← num_0 ⊕ num_1
l_u ?generated +2 ptr_0 ← shift num_0
guard i8 == 0
guard p2 != null
p43 = (p2: MetaStack).stack
p44 = (p2: MetaStack).tail
guard p43 == null
l_u ?generated +3 num_0 ← 0
l_u ?generated +4 ptr_1 ← l_0()
l_u ?generated +5 push stack ptr_0
...
l_u ?generated +12 return(ptr_0)
...
l_u ?generated +1 1 match {l_0() ⇒ l_8, l_*() ⇒ l_7}
guard p0 != null
i45 = (p0: Stack_arb_arb).target
guard i45 == 5
l_u ?generated +0 num_0 ← 0
...
l_u ?generated +6 jump l_6
l_u tnp441_569 +0 return()
i47 = (p0: Stack_3_1).values_num_0
i48 = (p0: Stack_3_1).values_num_1
i49 = (p0: Stack_3_1).values_num_2
p50 = (p0: Stack_3_1).values_ptr_0
p51 = (p0: Stack_arb_arb).tail
l_u ?generated +0 num_3 ← 0
l_u ?generated +1 num_3 ← primitive infixGt(num_2, num_3)
i53 = i49 > i53
guard i53
l_u ?generated +2 if num_3 == 0 then l_4()
l_u ?generated +3 num_3 ← 0
l_u ?generated +4 num_0 ← num_0 ⊕ num_3
l_u ?generated +5 num_3 ← 0
l_u ?generated +6 num_1 ← num_1 ⊕ num_3
l_u ?generated +7 num_3 ← 1
l_u ?generated +8 num_2 ← primitive infixSub(num_2, num_3)
i55 = i49 - i55
...
l_u ?generated +19 jump l_10
l_u bar_415 +0 push l_5(num_0, num_1, num_2, ptr_0)
l_u bar_415 +1 push l_9()
Label 5641224384:
l_u bar_415 +1 push l_9()
guard not invalidated
l_u bar_415 +2 num_0 ← 0
l_u bar_415 +3 num_0 ← num_1 ⊕ num_0
l_u bar_415 +4 ptr_1 ← l_0()
...
l_u bar_415 +8 ptr_1.m_0(num_0, ptr_0)
p56 = (p58: CoData_arb_arb).vtable
guard p56 == ConstPtr(ptr57)
i58 = (p58: CoData_2_2).values_num_0
i59 = (p58: CoData_2_2).values_num_1
p60 = (p58: CoData_2_2).values_ptr_0
p61 = (p58: CoData_2_2).values_ptr_1
l_u ?generated +0 num_1 ← 0
l_u ?generated +1 num_0 ← num_0 ⊕ num_1
l_u ?generated +2 ptr_0 ← shift num_0
guard i48 == 0
l_u ?generated +3 num_0 ← 0
l_u ?generated +4 ptr_1 ← l_0()
l_u ?generated +5 push stack ptr_0
...
l_u ?generated +12 return(ptr_0)
...
l_u ?generated +1 1 match {l_0() ⇒ l_8, l_*() ⇒ l_7}
l_u ?generated +0 num_0 ← 0
...
l_u ?generated +6 jump l_6
l_u tnp441_569 +0 return()
l_u ?generated +0 num_3 ← 0
l_u ?generated +1 num_3 ← primitive infixGt(num_2, num_3)
i64 = i55 > i64
guard i64
l_u ?generated +2 if num_3 == 0 then l_4()
l_u ?generated +3 num_3 ← 0
l_u ?generated +4 num_0 ← num_0 ⊕ num_3
l_u ?generated +5 num_3 ← 0
l_u ?generated +6 num_1 ← num_1 ⊕ num_3
l_u ?generated +7 num_3 ← 1
l_u ?generated +8 num_2 ← primitive infixSub(num_2, num_3)
i66 = i55 - i66
...
l_u ?generated +19 jump l_10
l_u bar_415 +0 push l_5(num_0, num_1, num_2, ptr_0)
l_u bar_415 +1 push l_9()
jump 5641224384

```

Fig. 6. Example trace

How we generated the simplified trace

- remove non-effect-overhead
- remove movement operations
- remove constant-foldable evidence computations
- remove region operations
- remove obviously unused allocations (will be removed)
- manually rename identifiers
- manually rewrite syntax

4.2.1 Linear handlers. We call a handler linear if it will always resume exactly once.

Argue informally, why the effect call overhead will always be optimized away.

4.3 Evaluation of the Approach

(Qualitative) Evaluation of Implementation

- Compare vs others regarding:
 - effort
 - * backend: reasonable (LOC? Rough time estimate?)
 - expressive power (if possible)
 - * All of Effekt-lifted
 - * Easily extends to almost anything
 - approach
 - * bytecode-vm with jit after compiling well-ish

5 CONCLUSION AND FUTURE WORK

TODO

Future Work

- Evaluate vs no-evidence jit
- More detailed loop heuristics

REFERENCES